

1. Executive Summary & Strategic Scope

Automated Geospatial Intelligence (GEOINT) plays a pivotal role in defense, border surveillance, and maritime domain awareness. Conventional computer vision frameworks cannot ingest massive 500MB+ overhead GeoTIFF images without destructive downsampling, eliminating sub-pixel signatures of critical threat assets.

The **Satellite Image Threat Detection (SITP)** platform v3.0.0 delivers a production-grade, end-to-end MLOps pipeline with zero downsampling through sliding-window chipping, batched NMS, **ONNX Runtime acceleration**, **smart background tile early-exit**, and a tactical web HUD with per-phase latency telemetry and high-threat intelligence briefs.

2. Multi-Tier System Architecture

Architecture Tier	Core Components	Primary Responsibilities & Outputs
Tier 1: Ingestion	DataIngestion xView_train.geojson	Parses raw geoJSON labels, verifies GeoTIFF rasters on disk, enforces leakage-free 80/20 train/val image-level splits.
Tier 2: Transformation	DataTransformation Albumentations	Slices large rasters into 512x512 chips with 100px overlap; converts coordinates to YOLO format; applies CLAHE and spatial augmentations.
Tier 3: Training & Diagnostics	ModelTrainer ModelMonitoring	Fine-tunes YOLOv8m backbone with CloU/DFL loss; tracks mAP50/mAP50-95 curves; runs per-class diagnostics.
Tier 4: ONNX Serving & HUD	PredictPipeline application.py (Flask)	ONNX Runtime inference engine with smart tile early-exit, per-phase latency tracker, high-threat brief generation, and dual-stream operational telemetry.

3. Latency Reduction & High-Throughput Engine (6 Techniques)

Large satellite images require evaluating dozens to hundreds of sliding-window tiles. SITP v3.0.0 implements six synergistic latency engineering techniques:

- ONNX Runtime Engine (best.onnx):** YOLOv8 weights exported to ONNX format (imgsz=512, dynamic=True, simplify=True) and served via ONNX Runtime 1.16+ with CPUExecutionProvider. Applies SIMD vectorization, graph fusion, and operator constant folding — delivering significant speedup over native PyTorch CPU.
- Smart Background Tile Early-Exit:** Before each 512x512 chip enters the network, a subsampled variance check (chip[:,4, :4]) rejects featureless ocean ($\sigma < 5.0$), void border padding ($\mu < 4.0$ and $\sigma < 3.0$), and cloud whiteout ($\mu > 245.0$ and $\sigma < 4.0$). Eliminates 25-60% of forward passes on background-heavy rasters.
- Batched Tile Forward Passes (batch_size=16):** Non-empty chips are vectorized into batches, achieving 3x-6x throughput speedup over sequential inference.
- In-Memory RAM Raster Slicing:** GeoTIFF multi-band array is read into RAM once via rasterio and normalized in memory, eliminating hundreds of repetitive disk seeks.
- Warm In-Memory Model Reuse:** PredictPipeline and Flask server retain the loaded model in RAM across web requests, eliminating per-inference model loading overhead.
- GPU FP16 Half-Precision Acceleration:** Automatically activates half-precision inference on CUDA-enabled hardware for orders-of-magnitude throughput improvement.

4. Tactical Web HUD — Result Panels

The Flask web dashboard at <http://localhost:5000> renders eight data panels in the results view after each analysis:

Panel	Data Displayed	Source Variable
Stats Bar	Threats found, avg confidence, unique classes, inference time, tiles skipped	total_detections, avg_confidence, latency_tracker
Engine Badge	Active engine (ONNX Runtime / PyTorch) and tiles processed count	engine_type, latency_tracker.tiles_processed
High-Threat Brief	Natural-language summary of detections >= 75% confidence with pill tags per class	threat_brief, high_threat_objects, high_threat_total
Latency Breakdown	4 animated color-coded progress bars: Raster Load, Forward Pass, NMS, Render	latency_tracker.phases[]
Image Metadata	Filename, resolution (WxH), bands, file size, driver format	img_metadata
Detection Overlay	Base64 JPEG with annotated bounding boxes and class labels	result_image
Class Breakdown	Horizontal bar chart of all detected classes sorted by count	class_breakdown[]
Detection Log	Full itemized table: #, class name, confidence badge, x1 y1 x2 y2	detections[]

5. Dual Logging & Telemetry Architecture

SITP establishes a clean dual-logger infrastructure separating operational system telemetry from analytical detection records:

Log Stream	Target Destination	Captured Telemetry Content
Backend & Deployment Logger	logs/backend/backend_*.log	Server lifecycle, health checks (/health), model checkpoint loading, route requests, ONNX/PyTorch engine selection, training pipeline orchestration, and system exceptions.
Detection Details Logger	logs/detection/detection_*.log	Dedicated threat detection records: GeoTIFF dimensions, file size, confidence parameters, engine type, per-phase latency breakdown (load/infer/nms/render ms), tiles processed/skipped, class breakdown percentages, high-threat count, and itemized bounding boxes.

6. Deployment Topology & Container Architecture

The platform is fully containerized via Dockerfile based on a hardened Python 3.10+ runtime. The Docker build automatically exports best.pt to best.onnx if not already present (via ultralytics YOLO.export(format='onnx', imgsz=512, dynamic=True, simplify=True)). The application dynamically selects the faster ONNX engine at startup. REST endpoints (GET /, POST /, GET /health) support standalone deployment, Kubernetes cluster auto-scaling, and edge container orchestration.

7. Security, Memory Safety & Failure Recovery

SITP enforces: (1) UUID-isolated file handling with automatic cleanup in finally blocks, (2) Strict file extension verification (.tif, .tiff), (3) Centralized traceback enrichment via CustomException, (4) ONNX Runtime verbose log suppression via ORT_DISABLE_ALL_LOGS=1, and (5) In-memory array bounding with garbage collection.